Lecture 15 - Routing: link state and Dijkstras Algorithm

→ every router has a forwarding table → those prefixes are CIDR blocks from hierarchical address allocation

→ <u>how does the forwarding table get built?</u>
Static routing: a human configures every entry. Does not scale
Dynamic routing: routers automatically computer tables and adapt when the network changes

2 major families:
- Link state: every router learns the full topology and computes the shortest path
    - Location information distributed globally
- Distance vector: routers learn only from neighbors and converge iteratively
    - Global information distributed locally

The graph abstraction
Computer network maps directly to graph G = (V,E)
- V → set of vertices / nodes → routers
- E → set of edges / links → links between routers

Graph term	Networking term
Vertex / node	Router
Edge / link	Physical or logical connection
Adjacent	Directly connected
Degree	Number of links on a router
Path	Sequence of hops

Edge weight / costs
Weighted graphs assignes a numerical cost to each edge → cost determines what the shortest path is

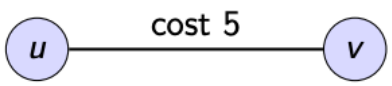
Weight meaning	Lower is...	Used by
Hop count	Fewer routers traversed	RIP (all costs = 1)
Inverse bandwidth	Higher-capacity link	OSPF default: $10^8/B$
Propagation delay	Lower latency	Delay-sensitive routing
Administrative cost	Preferred by policy	Operator-configured

OSPF default cost →

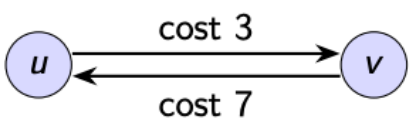
Link type	Bandwidth	OSPF cost	tx time of 100 kb
T1	1.544 Mb/s	64	64 ms
10 Mb/s Ethernet	10 Mb/s	10	10 ms
100 Mb/s Fast Ethernet	100 Mb/s	1	1 ms

Directed vs undirected

Undirected edge/ arcs → asymmetric → different cost per direction → most full duplex links



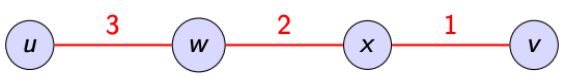
Directed edge → symmetric → same cost both ways → exemple: satellite where uplink is slow and downlink is fast



<u>→ primarily use undirected graphs</u>

Paths, path cost, shortest paths

Path from u to v is a sequence of vertices connected by edges:



$\text{cost}(u \to w \to x \to v) = c(u,w) + c(w,x) + c(x,v) = 3 + 2 + 1 = 6$ --> cost function is c(edge1,edge2)

shortest path from u to v has the minimum total cost

- Adjacent/ neighbots = share and edge
- Degree = number of edges on a node
- Cycle = path that starts and ends at the same vertex

Routing problem → find the shortest path from each router to every destination

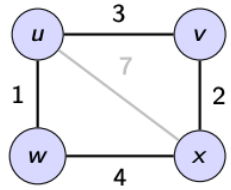
Trees and shortest path trees

<u>Tree</u> = a connected graph with no cycles

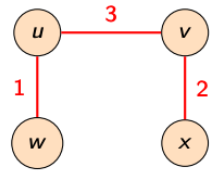
<u>Spanning tree</u> = subgraph that includes all vertices and exactly |V|-1 edges

<u>Shortest path tree (SPT)</u> = rooted at u=spanning tree where the path from u to every other vertex is the shortest path in the original graph

Original graph (has cycles):



SPT rooted at u:



3 edges, 4 nodes, no cycles

→ dijkstras algorithm computes and SPT → SPT is exactly what a router needs to build its forwarding table

Link state routing

Link state approach → every router gets a compete map → each computes shortest paths independently

→ analogy : having a full road map and computing your own best route

Distance vector approach → each router knows only itd direct neighbors and their distance estimates

→ analogy : asking locals for directions at each intersection

Link state process has 3 phases →

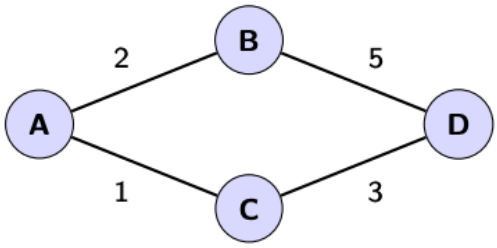
After flooding: every router has the same topology database

Dijkstra is deterministic → all routers compute consistent forwarding tables

Phase 1-2: discover and flood

Phase 1: each router sends hello packets on all interfaces and discovers neighbors and links cost → link state advertisement (LSA)

Phase 2: each router floods its LSA to all other routers



After flooding, every router knows:

A's LSA: {A–B: 2, A–C: 1}
B's LSA: {B–A: 2, B–D: 5}
C's LSA: {C–A: 1, C–D: 3}
D's LSA: {D–B: 5, D–C: 3}

Same data at every router.

Flooding rules to prevent infinite loops

- Each LSA has a sequence number → routers discard already seen LSAs
- Forward received LSA out all interfaces except the one it arrived on

- <u>LSAs have a maximum age</u> → refreshed periofically

Phase 3: dijkstras algorithm

given : weighted graphs G(V,E) and source node s

Find: shortest path from s to every other node

Properties:

- D(v) → current best-known cost from s to v
- p(v) → predecessor of v on the best-known path
- N ′ → the set of nodes whose shortest path is finalized

Relaxation and why dijkstra

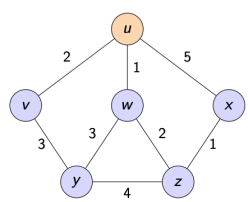
<u>Relaxation → testing whether a path through u improves the cost to v:</u>

If D(u) + c(u, v) < D(v), then set D(v) = D(u) + c(u, v)

Why is the greedy choice correct?

→ D(w) is the true shortest-path cost → this is why negative weights break Dijkstra

Example



Source node: u (orange). Six nodes, eight edges.

Edge	u–v	u–w	u–x	v–y	w–y	w–z	x–z
Cost	2	1	5	3	3	2	1

Edge	y–z
Cost	4

Initialization: $N' = \{u\}$, set costs to direct neighbors, ∞ for rest.

Step	N'	$D(v), p(v)$	$D(w), p(w)$	$D(x), p(x)$	$D(y), p(y)$	$D(z), p(z)$	Select
Init	$\{u\}$	2, u	1, u	5, u	∞	∞	—

Step 1: Minimum is w ($D = 1$). Add w to N'. Update w's neighbors:

- y: $D(w) + c(w, y) = 1 + 3 = 4 < \infty \Rightarrow D(y) = 4$, $p(y) = w$
- z: $D(w) + c(w, z) = 1 + 2 = 3 < \infty \Rightarrow D(z) = 3$, $p(z) = w$

Step	N'	$D(v)$	$D(w)$	$D(x)$	$D(y)$	$D(z)$	Select
Init	$\{u\}$	2	1	5	∞	∞	—
1	$\{u, w\}$	2	**1**	5	4	3	w

Step 2: Minimum is v ($D = 2$). Add v to N'. Update:

- y: $D(v) + c(v, y) = 2 + 3 = 5 > 4 \Rightarrow$ no change (path through w is better)

Step 3: Minimum is z ($D = 3$). Add z to N'. Update:

- x: $D(z) + c(z, x) = 3 + 1 = 4 < 5 \Rightarrow D(x) = 4$, $p(x) = z$ **improvement!**
- y: $D(z) + c(z, y) = 3 + 4 = 7 > 4 \Rightarrow$ no change

Step	N'	$D(v)$	$D(w)$	$D(x)$	$D(y)$	$D(z)$	Select
Init	$\{u\}$	2	1	5	∞	∞	—
1	$\{u, w\}$	2	**1**	5	4	3	w
2	$\{u, w, v\}$	**2**	—	5	4	3	v
3	$\{u, w, v, z\}$	—	—	$5 \to 4$	4	**3**	z

Step 3 discovered a better path to x: $u \to w \to z \to x$ (cost 4) beats the direct path $u \to x$ (cost 5).

Step 4: Tie between x and y (both $D = 4$). Pick x. No improvements.

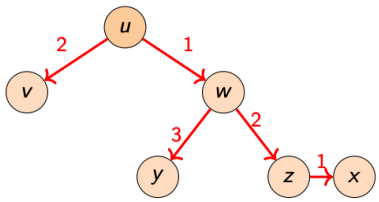
Step 5: y is the last node. $D(y) = 4$, $p(y) = w$. Done.

Step	N'	$D(v)$	$D(w)$	$D(x)$	$D(y)$	$D(z)$	Select
Init	$\{u\}$	2	1	5	∞	∞	—
1	$\{u, w\}$	2	**1**	5	4	3	w
2	$\{u, w, v\}$	**2**	—	5	4	3	v
3	$\{u, w, v, z\}$	—	—	$5 \to 4$	4	**3**	z
4	$\{u, w, v, z, x\}$	—	—	**4**	4	—	x
5	$\{u, w, v, z, x, y\}$	—	—	—	**4**	—	y

Final shortest-path costs from u: v: 2 w: 1 z: 3 x: 4 y: 4

The shortest path tree

From the predecessor pointers p(v) → reconstruct the shortest path tree rooted at u



5 edges, 6 nodes, no cycles

Shortest paths from u:

Dest	Path	Cost
v	$u \to v$	2
w	$u \to w$	1
y	$u \to w \to y$	4
z	$u \to w \to z$	3
x	$u \to w \to z \to x$	4

Note: direct link $u \to x$ (cost 5) is **not** in the SPT. The indirect path through w and z is cheaper (cost 4).

SPT to forwarding tree

SPT tells router u to reach destination which first hop should i use ?

Destination	Next hop	Cost	Path
v	v	2	$u \to v$
w	w	1	$u \to w$
x	w	4	$u \to w \to z \to x$
y	w	4	$u \to w \to y$
z	w	3	$u \to w \to z$

--> SPT structure determines the forwarding table and the router only needs to know the first hop , not the full path

Every router runs Dijkstra

In link state network with N routers

- Every router has the same topology database from flooding
- Each router runs dijkstras rooted at itself
- Each produces a different SPT

When do routers recompute?

1. A new LSA arrives
2. A LSA ages out and is refreshed
3. A new router joins the link

In stable networks → Dijkstra runs infrequently → so the forwarding table stays static

Convergence

Convergence = all routers agreeing on consistent forwarding tables after a change. Link-state converges fast (seconds) because flooding is fast and each router independently computes a correct result from the complete topology.

OSPF: open shorts path first → most widely deployed link state routing protocol

Implements exactly the 3 phase process

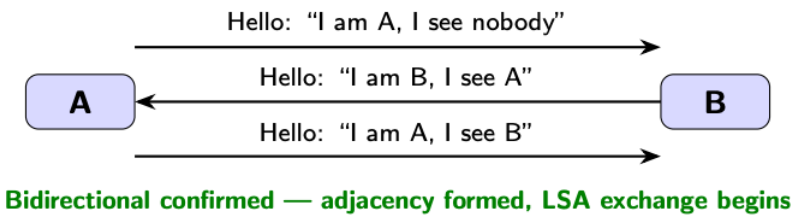
1. Hello packets → discover neighbors (default: every 10s on broadcast links)
2. LSA flooding → build the link-state database (LSDB) at every router
3. SPF computation → Dijkstra on the LSDB → forwarding table

Key details:

- Runs directly over IP (protocol number 89 — not TCP, not UDP)
- Dead interval: if no hello received for 40s, neighbor declared down → new LSA flooded

- RFC 2328 (OSPFv2 for IPv4), RFC 5340 (OSPFv3 for IPv6)
- IS-IS is the other major link-state protocol (used by large ISPs)

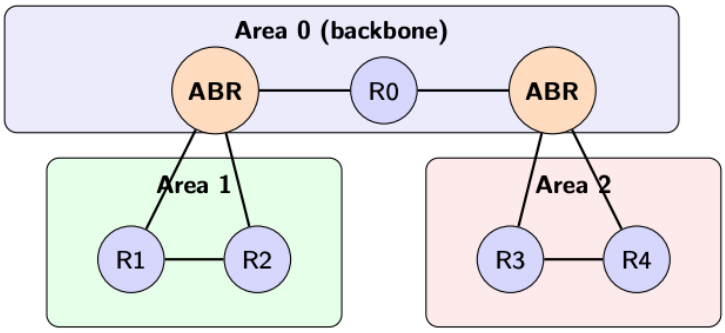
OSPF hello protocol



3 purposes:

1. Neighbor discovery → find adjacent routers
2. Bidirectional verification → confirm the neighbor sees you too
3. Failure detection → if hellos stop (dead interval = 40s), declare neighbor down

OSPF areas → In large networks (hundreds of routers), flooding everywhere is expensive → OSPF uses areas to contain flooding and reduce LSDB size



- Area 0 = backbone; all other areas must connect to it
- ABR (Area Border Router) connects areas and summarizes routes between them
- LSA flooding is contained within an area — smaller LSDB, faster Dijkstra

OSPF packet types → OSPF defines five packet types

Key fields in an OSPF packet:

Type	Name	Purpose
1	Hello	Neighbor discovery, keep-alive
2	DB Description	Summarize LSDB during initial sync
3	LS Request	Request specific LSAs from neighbor
4	LS Update	Carry one or more LSAs
5	LS Acknowledgment	Confirm receipt (reliable flooding)

<u>Router ID:</u> 32-bit identifier (often highest IP on the router)
<u>Area ID:</u> 0.0.0.0 for backbone
<u>LSA age:</u> counts up from 0; max age = 3600s (1 hour) → LSA flushed
<u>LS Update</u>: contains the actual link costs → the raw data for Dijkstra

The oscillation problem

<u>Traffic shifts → costs change → path changes → traffic shifts again → never stabilizes</u>

Solutions:

1. Don’t use load-sensitive costs → OSPF/IS-IS use static costs based on bandwidth → This is standard practice
2. Dampen changes with exponentially weighted moving averages
3. Randomize recomputation times to prevent lock-step oscillation